

From Entropy to Epiplexity

Rethinking Information for Computationally Bounded Intelligence

Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J. Zico Kolter, Andrew Gordon Wilson

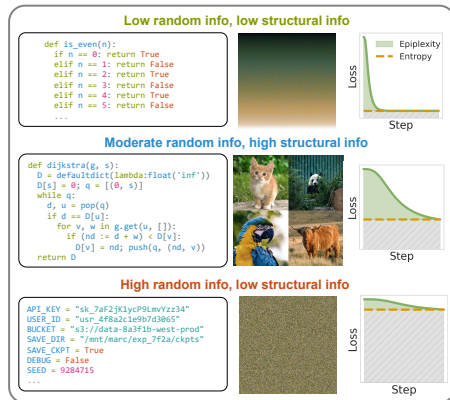
Carnegie Mellon University, New York University

arXiv:2601.03220, version 2, 16 March 2026

Speaker: Alex Kravatsky

The question: which data teach a model?

- Pretraining on text transfers to robotics control, theorem proving and forecasting. The same number of image bytes transfers less.
- A dataset can contain many unpredictable bits and still teach a useful, reusable pattern.
- **Question:** how much structure can a particular learner extract, given a fixed compute budget?



Training loss on three kinds of data. Dashed line: final loss; green area: loss removed by learning. Source: Fig. 1 of Finzi et al. (2026).

Two classical lenses on information

$H(X) = \mathbb{E}[-\log_2 p(X)]$ is the Shannon entropy of a random variable X with probability mass function p . It is the average number of bits needed to encode a draw from the distribution.

$K(x)$ is the Kolmogorov complexity of one realised string x : the length of the shortest program that prints x on a fixed universal computer $\mathcal{U}$ (Li and Vitányi, 2008).

Shannon entropy

Uncertainty in a distribution. Uniform random bits have high entropy; a deterministic alternating string has low entropy.

Kolmogorov complexity

A shortest executable recipe. An alternating string has a short recipe; a typical random string does not.

Kolmogorov complexity is a mathematical benchmark; no algorithm computes $K(x)$ exactly for every string.

These are complementary: entropy describes average uncertainty, while Kolmogorov complexity describes one object's shortest recipe.

The missing ingredient: computational access

Neither classical quantity asks whether a learner can find or use the relevant pattern within a finite budget. For example, a deterministic training run can produce a network with useful circuits, although the rules and random seed form a short description of the run.

- A weak observer may treat a structured process as random.
- A stronger observer may discover a compact predictive model.
- We therefore define information relative to the models available within time T .

Epiplexity is the structure that a computationally bounded observer can extract.

A model with bounded computation

Fix a prefix-free universal computer $\mathcal{U}$ and a time bound $T(n)$. The observer may use only programs $P \in \mathcal{P}_T$ that, within $T(n)$ steps, can both

- evaluate a probability $P(x)$ for an observation x ;
- sample a new x from the same distribution P .

The program length $|P|$ is measured in bits. This is the idealised definition; in experiments we replace $\mathcal{P}_T$ by models reachable with a stated architecture, optimiser and FLOP budget.

The time bound describes what the observer can compute, not a property of the data alone.

Epiplexity: the two-part description

Definition 1 (Def. 8 of the paper)

For a random variable X on $\{0, 1\}^n$, let

$$P^* = \arg \min_{P \in \mathcal{P}_T} \{ |P| + \mathbb{E}[-\log P(X)] \},$$

with ties broken by the shortest program. Logarithms are base 2. The *epiplexity* of X is $S_T(X) := |P^*|$. The *time-bounded entropy* of X is $H_T(X) := \mathbb{E}[-\log P^*(X)]$.

S_T is the model part: bits needed to describe the useful predictor. H_T is the data part: bits that the predictor cannot explain. Their sum $\text{MDL}_T(X) := S_T(X) + H_T(X)$ is the minimum two-part description length under the bound.

Random variable X	Time bound	$S_T(X)$	$H_T(X)$
U_n , uniform on $\{0, 1\}^n$	$T(n) \geq c_1$	$\leq c_2$	$\geq n$
0101... or 1010..., probability $\frac{1}{2}$ each	$T(n) = \Theta(n)$	$O(1)$	$O(1)$

For U_n , $S_T = O(1)$ and $H_T \geq n$. For the alternating pair, both parts are $O(1)$.

Two consequences of the time bound

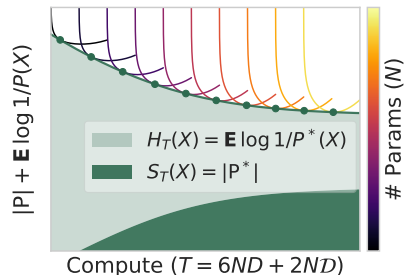
- **Pseudorandom generator.** A short seed is stretched into n bits. Shannon entropy sees only the seed; a polynomial-time observer sees almost n unpredictable bits and almost no structure.
- **One-way map.** For $Y = f(X)$, predicting Y from X is easy, while recovering X from Y is hard. A bounded observer assigns different information to the two orders.

For supervised learning we use the conditional version $S_T(Y \mid X)$: the input X is available, and only the labels Y are encoded.

The same data can look random or structured depending on what the observer can compute.

From programs to networks: the compute-optimal code

- Search over all programs is intractable, so fix an architecture and optimiser.
- Compute accounting: $6ND + 2ND \leq T$ for training and evaluation.
- Each run gives a model code plus a validation-data code. The lower convex hull over N and D estimates MDL_T .
- At compute T , split the hull into S_T (model) and H_T (data).

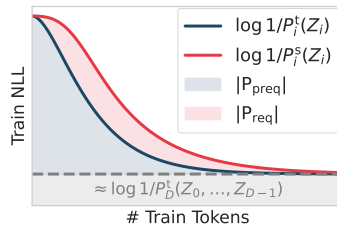


Schematic. Thin curves: training runs, coloured by N . Dots: the lower envelope. Dark green: S_T ; light green: H_T . Source: Fig. 2b of Finzi et al. (2026).

Estimation I: the prequential code

- Train P_i on the first i examples of an i.i.d. stream.
- Encode example Z_i with $-\log_2 P_i(Z_i)$ bits, then train on it.
- Subtract the final-model floor. The remainder is the area above the final loss:

$$|P_{\text{preq}}| \approx \sum_{i < M} [\log 1/P_i(Z_i) - \log 1/P_M(Z_i)].$$



Dark curve: $\log 1/P_i(Z_i)$ of the network trained on real data (superscript t). Blue area: $|P_{\text{preq}}|$. Red curve and red area: the prequential code of the next slide.
Source: Fig. 2a of Finzi et al. (2026).

- The loss is measured before fitting each example, so it is a test-like loss.
- This model-part estimate is heuristic; decoding-time guarantees are not established.

Estimation II: the sequential code

- A teacher trains on real data; a student trains only on teacher samples. Encoder and decoder share the student (Qiu et al., 2026).
- Relative entropy coding pays for their disagreement, $KL_i = KL(P_i^t \parallel P_i^s)$:
- The code length of the final student is

$$|P_{\text{req}}| = \sum_{i < M} [KL_i + \log(1 + KL_i) + 4] + O(1) \approx \sum_{i < M} KL_i.$$

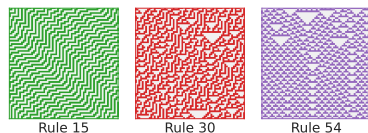
	Prequential	Requential
Code	heuristic, decoding time unknown	explicit, decoded in $6ND$ FLOPs
Training	one network	teacher and student, 2–10× slower
Estimate	several times larger	smaller

Prequential is cheaper; requential has an explicit code.

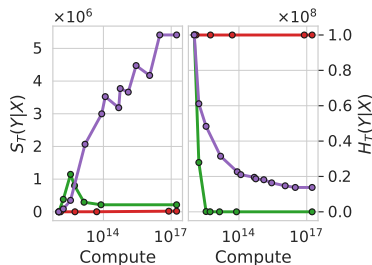
Experiment: cellular automata create structure

An *elementary cellular automaton* (ECA) updates binary cells in parallel from their three-cell neighbourhoods (Wolfram, 2002).

Setting. 64 periodic cells; $Y = F_r^{48}(X)$; transformer sweep; sequential code.



Rule	15	30	54
$S_T(Y X)$, bits	2×10^5	≈ 0	5.4×10^6
$H_T(Y X)$, bits	≈ 0	10^8	1.5×10^7

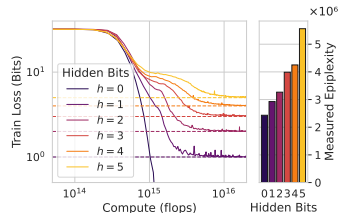
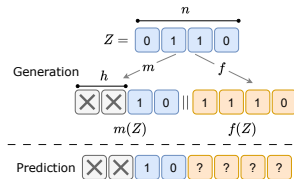


Left: rollouts, time downward. Right: S_T and H_T against compute in FLOPs. Table: our estimates from the plots at 10^{17} FLOPs. Source: Fig. 3 of Finzi et al. (2026).

Rule 15 is simple, rule 30 looks random, and rule 54 contains learnable structure.

Experiment: a model learns more than the generator contains

Setting. Hide h bits of a random $Z \in \{0,1\}^{32}$; predict four rule 30 updates of Z from the visible bits.



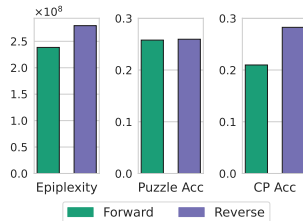
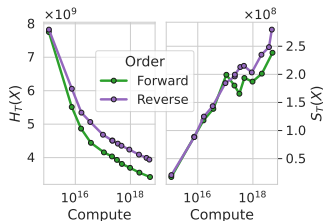
Left: the data. Right: training loss against compute in FLOPs (dashed: h bits) and S_T for each h . Source: Fig. 5 of Finzi et al. (2026).

- The generator runs forward; the predictor must infer the hidden bits.
- The loss approaches h bits only after rapidly growing compute, and S_T grows with h .

Under a time bound, maximum likelihood learns an inference circuit that the generating program does not contain.

Experiment: the order of chess data

Setting. Lichess games in two orders: moves $\rightarrow$ final board, or board $\rightarrow$ moves. Transformer sweep with sequential coding.



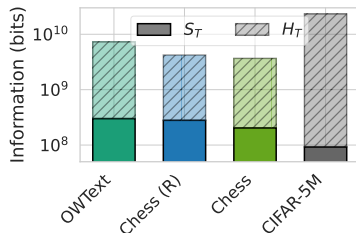
Left: H_T and S_T against compute in FLOPs. Right: S_T and accuracy after fine-tuning. Source: Figs. 4c and 7 of Finzi et al. (2026).

- At large compute, reverse order gives higher H_T and S_T .
- It transfers better to board evaluation: 0.28 vs 0.21 accuracy; puzzles are tied.

The reverse order forces a model of the board state. That structure transfers to a task on board states.

Application: structure in natural data

Setting. Equal token budgets and a shared compute bound. Compare OpenWebText, chess and CIFAR-5M with the sequential code.



Chess (R): reverse order. Source: Fig. 8a of Finzi et al. (2026). Table: our estimates from the figure.

Dataset	S_T , bits	MDL_T , bits	S_T/MDL_T
OpenWebText	3×10^8	7×10^9	4%
Chess, reverse	2.8×10^8	4×10^9	7%
Chess, forward	2×10^8	3.5×10^9	6%
CIFAR-5M	9×10^7	2.3×10^{10}	0.4%

- Text has the most structure; pixel images have the most residual information.
- Scaling laws and adaptive data selection show the same ordering (Jiang et al., 2025).

Images contain the most information and the least structure. Text contains the most structure.

Summary and limitations

Summary

- S_T is the model bits; H_T is what that model cannot predict.
- A time bound makes computation, prediction order and learned structure observable.
- Estimate S_T with prequential area or teacher–student divergence at the compute-optimal model.

Limitations

- S_T is not a guarantee of downstream usefulness.
- The theorems are asymptotic and cryptographic; the proved lower bound is only $\Omega(\log n)$.
- Estimates depend on architecture, optimiser, T , dataset size and random seed.

Literature

M. Finzi et al. *From Entropy to Epilepsy*. arXiv:2601.03220v2, 2026.

S. Qiu et al. *Requential Coding*. arXiv:2607.11883, 2026.

M. Li, P. Vitányi. *Kolmogorov Complexity*. Springer, 2008.

S. Wolfram. *A New Kind of Science*. 2002.

Y. Jiang et al. Adaptive data optimization. *ICLR*, 2025.

Questions for the audience

1. A dataset X consists of n uniformly random bits. Which quantity is close to n : the epiplexity $S_T(X)$ or the time-bounded entropy $H_T(X)$?
2. In the prequential estimate, which part of the training loss curve approximates the epiplexity?

Backup: basic properties

Proposition 1 (Sec. 3 of the paper)

Let c_1, c_2 be constants independent of X . For a random variable X on $\{0, 1\}^n$:

1. $S_T(X) \geq 0$ and $H_T(X) \geq 0$;
2. $H(X) \leq \text{MDL}_T(X) \leq n + c_1$;
3. $\text{MDL}_{T'}(X) \leq \text{MDL}_T(X)$ whenever $T'(n) \geq T(n)$;
4. $\text{MDL}_{T'}(f^{-1}(X)) \leq \text{MDL}_T(X) + |f| + c_2$ with $T'(n) = T(n) + \text{Time}(f)$, for a program f that computes a bijection f in fixed time.

Property 4 is the analogue of $K(f(x)) \leq K(x) + K(f) + c$. Under a time bound a short program for f^{-1} gives no short program for f .

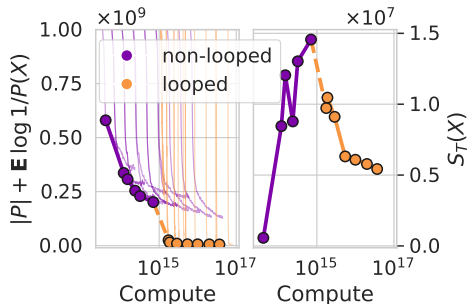
Theorem 1 (Thm. 10 of the paper)

If one-way functions secure against probabilistic polynomial-time algorithms with polynomial advice exist, then some random variables X_n on $\{0, 1\}^n$ have $S_{\text{Poly}}(X_n) = \Omega(\log n)$.

Backup: emergence, where S_T falls as T grows

Setting. ECA rule 54, 64 updates. A non-looped transformer predicts the final state directly. A looped transformer generates the intermediate states one after another: the exact simulation.

- Below 10^{15} FLOPs the non-looped model gives the shorter code. The epiplexity of this model rises to 1.45×10^7 bits: it learns emergent patterns of rule 54.
- Above that bound the looped model gives the shorter code. MDL_T drops to almost zero; S_T falls.
- The authors call this case uncommon. For natural data at moderate compute they expect S_T to grow with compute.



Two-part code and S_T against compute in FLOPs.
Source: Fig. 6 of Finzi et al. (2026).

With less compute the best model is a longer program: S_T need not grow with T .